axiateam
RDT company

Módulo 11

Importaciones y Exportaciones

Curso: TypeScript

75 min

Objetivos de Aprendizaje

- Dominar diferentes tipos de **exports** (named, default, re-export)
- Implementar **imports** con alias
- Utilizar **barrel exports**
- Comprender **ES modules vs CommonJS**

1. Contexto

Sistemas de Módulos

JavaScript Modules History

- CommonJS: `require()` / `module.exports` (Node.js)
- AMD: `define()` / `require()` (histórico)
- ES Modules: `import` / `export` (estándar moderno)

TypeScript soporta todos, usaremos ES Modules.

2. Exports

Tipos de Export

Named exports para varios elementos, default export para uno principal.

```
// Named exports - varios
export const API_URL = "https://api.example.com";
export type TaskStatus = "pending" | "completed";
export interface Task { id: string; title: string; }

// Default export - uno principal
export default class TaskService {
  // ...
}

// Re-export desde otro módulo
export { Task } from "./types";
export { TaskService } from "./taskService";
```

Lab: Exports Organizados

Objetivo: Implementar exports limpios para el sistema de tareas.

1. Crea exports en `src/types/index.ts`
2. Crea exports en `src/models/index.ts`
3. Crea exports en `src/services/index.ts`

Código del Lab

```
// src/types/index.ts
export type { Task, TaskStatus, TaskPriority } from "./task";
export { TaskStatus, TaskPriority } from "./enums";
export type { Filterable } from "./interfaces";

// src/models/index.ts
export { TaskModel } from "./task";
export { UserModel } from "./user";

// src/services/index.ts
export { TaskService } from "./taskService";
export { UserService } from "./userService";
```

3. Imports

Tipos de Import

Los imports pueden ser named, con alias, default o combinados.

```
// Named import
import { Task, TaskStatus } from "./types";

// Renombrar con alias
import { Task as TaskEntity } from "./types";

// Default import
import TaskService from "./TaskService";

// Todo como objeto
import * as Utils from "./utils";

// Combinar default y named
import React, { useState, useEffect } from "react";
```

Lab: Imports con Alias

Objetivo: Usar imports limpios y organizados.

1. Importa con alias para evitar conflictos
2. Combina múltiples exports
3. Usa import * as para utilidades

Código del Lab

```
// Alias para evitar conflictos
import { Task as TaskEntity } from "@/types";
import { Task as TaskDTO } from "@/api";

// Múltiples exports
import { TaskService, UserService } from "@/services";

// Todo como objeto
import * as Utils from "@/utils";
console.log(Utils.formatDate, Utils.formatString);

// Combinar default y named
import TaskService, { Task, TaskStatus } from "@/services";
```

4. Barrel Exports

Barrel Pattern

Un "barrel" es un archivo que re-exporta todo de un directorio:

```
// src/models/index.ts (barrel)
export { TaskModel } from "./task";
export { UserModel } from "./user";
export { BookModel } from "./book";

// Ahora puedes importar desde un solo lugar
import { TaskModel, UserModel, BookModel } from "@/models";
```

Lab: Crear Barrels

Objetivo: Implementar barrel exports para organización.

1. Crea barrel en `src/types/index.ts`
2. Crea barrel en `src/services/index.ts`
3. Verifica uso con un solo import

Código del Lab

```
// src/types/index.ts (barrel)
export * from "./task";
export * from "./enums";
export * from "./interfaces";

// src/services/index.ts (barrel)
export * from "./taskService";
export * from "./userService";

// Un solo import para todo el módulo
import {
  Task,
  TaskStatus,
  TaskPriority,
  TaskService,
  UserService
} from "@/types";
```

Resumen

Puntos Clave

- **Named exports**: múltiples por archivo
- **Default export**: uno principal por archivo
- **Re-export**: combina y re-expande módulos
- **Barrel**: archivo index que re-exporta directorio
- **Alias**: renombra con `as`

Ejercicio Integrador

Sistema de Biblioteca

Caso 2: Biblioteca

Objetivo: Aplicar al Caso 2.

1. Crea barrels para cada dominio
2. Implementa re-exports
3. Usa alias para evitar conflictos

Trabajo en grupos de 2-4 personas.

Tips y Gotchas

✅ Tips

- Named exports: preferibles para tree-shaking
- Default export: uno por archivo, para módulos principales
- Barrel: re-export desde index.ts

⚠️ Gotchas

- Default export dificulta refactoring
- Re-exports no tree-shakean automáticamente
- `import type` solo importa tipos, no código runtime

Referencias

Recursos Adicionales

- Export
- Import

Gracias!

Módulo 11 Completado

Siguiente: Módulo 12 - Testing con TypeScript